

Graph Coloring

Tascu Stelian-Andrei, Hutu Matei-Alexandru and Plesa Marian-Cosmin

Facultatea de Automatica si Calculatoare
321CA

Această lucrare explorează metodele de colorare a grafurilor, inclusiv algoritmi exacti precum backtracking și abordări euristice precum algoritmul greedy. Sunt analizate complexitatea și performanța acestora, utilizând seturi de teste variate. Eficiența acestora depinde de natura și de caracteristicile inputului. Obiectivul este de a înțelege compromisurile algoritmice și de a identifica soluții eficiente pentru scenarii din lumea reală, subliniind importanța colorării grafurilor în optimizarea resurselor și rezolvarea problemelor complexe.

1 Introducere

1.1 Descrierea problemei

Problema **Colorarea Grafurilor** reprezintă o provocare fundamentală în teoria grafurilor și are aplicații diverse în domenii precum planificarea, alocarea resurselor și optimizarea. Aceasta constă în atribuirea unei culori fiecărui nod al unui graf neorientat astfel încât orice două noduri adiacente (conectate printr-o muchie directă) să nu aibă aceeași culoare. Colorarea care utilizează cel mult culori se numește k -colorare, iar numărul minim de culori necesar pentru a colora un graf se numește numărul cromatic al grafului.

Istoria problemei: Problema colorării grafurilor își are originile în secolul al XIX-lea, în contextul colorării hărților geografice. În 1852, Francis Guthrie a formulat pentru prima dată întrebarea: „Care este cel mai mic număr de culori necesar pentru a colora o hartă astfel încât regiunile adiacente să aibă culori diferite?” Această întrebare a condus la dezvoltarea teoremei celor patru culori, care afirmă că orice hartă poate fi colorată cu cel mult patru culori. Deși enunțată inițial ca o conjectură, teorema a fost demonstrată abia în 1976 de Kenneth Appel și Wolfgang Haken, folosind o demonstrație asistată de calculator. Ulterior, demonstrația a fost formalizată utilizând sistemul Coq de Georges Gonthier.

Colorarea grafurilor a fost ulterior enunțată în teoria grafurilor, extinzând aplicațiile problemei dincolo de colorarea hărților. De exemplu, problema își găsește aplicabilitatea și în alocarea registrelor unui procesor, planificarea examenelor sau optimizarea canalelor de comunicare.

În 1981, Gregory Chaitin a demonstrat echivalența dintre colorarea grafurilor și problema register allocation (alocarea registrelor) în compilatoare. Această

aplicație practică a condus la dezvoltarea algoritmului Chaitin-Briggs în 1992, care este utilizat pe scară largă în compilatoare moderne, inclusiv GCC.

1.2 Aplicații în realitate

Problema colorării grafurilor are numeroase aplicații practice în diverse domenii, incluzând:

Planificări: Problema colorării grafurilor este folosită pentru a modela, optimiza și rezolva numeroase probleme de programare a sarcinilor. În forma sa generală, se dorește atribuirea unor intervale de timp unui set de sarcini, astfel încât să se evite conflictele. Fiecare sarcină necesită un interval, iar sarcinile care intră în conflict – de exemplu, pentru că utilizează aceeași resursă – nu pot fi programate simultan. Problema poate fi reprezentată printr-un graf, unde nodurile reprezintă sarcinile, iar muchiile indică conflicte între perechi de sarcini. [Marx, Dániel (2004), "Graph colouring problems and their applications in scheduling", Periodica Polytechnica, Electrical Engineering, vol. 48, pp. 11–16,]

Numărul cromatic al grafului corespunde timpului optim necesar pentru finalizarea tuturor sarcinilor fără conflicte. Astfel, colorarea grafului oferă o soluție optimă pentru programarea sarcinilor în intervale de timp distincte. Detaliile specifice ale problemei definesc structura grafului rezultat:

- În cazul alocării aeronavelor pentru zboruri, graful conflictelor rezultat este un **graf de intervale**, ceea ce face ca problema colorării să poată fi rezolvată eficient.

- În alocarea frecvențelor pentru stațiile radio, graful conflictelor este un **graf de discuri unitare**, iar problema colorării are soluții aproximative eficiente (3-approximabile).

Aceste aplicații ilustrează modul în care teoria grafurilor, prin colorarea acestora, poate optimiza programarea sarcinilor și utilizarea resurselor în sisteme complexe.

Sudoku: Problema Sudoku poate fi reprezentată ca o problemă de colorare a grafurilor. Fiecare celulă a tablei Sudoku este modelată ca un nod, iar două noduri sunt conectate printr-o muchie dacă celulele corespunzătoare sunt în același rând, coloană sau submatrice. Scopul este de a colora nodurile grafului folosind numerele de la 1 la 9, astfel încât două noduri conectate printr-o muchie să nu aibă aceeași culoare.

Modul de funcționare:

1. **Construcția grafului:** Graful are 81 de noduri (pentru un Sudoku 9x9), iar fiecare nod este conectat cu până la 20 de vecini.

2. **Pre-colorare:** Unele noduri sunt deja colorate, corespunzător numerelor date inițial.

3. **Algoritm de colorare:** Se utilizează algoritmi precum backtracking pentru a colora nodurile rămase respectând restricțiile. (Lewis (2021), pp. 172–179, Section 6.4: Latin squares and sudoku puzzles.)

2 Demonstrație NP-Hard

Pentru a demonstra că problema **Graph Coloring** este NP-Hard, vom transforma o problemă deja cunoscută ca fiind greu de rezolvat, **3-SAT**, într-o problemă de colorare a grafurilor.

2.1 Ce este 3-SAT?

3-SAT este o problemă logică în care trebuie să verificăm dacă o listă de reguli este adevărată. Fiecare regulă spune că **cel puțin una dintre trei afirmații este adevărată**.

- Avem variabile care pot fi fie adevărate (*True*), fie false (*False*). - Fiecare regulă are exact trei afirmații despre variabile. De exemplu: - "Variabila x_1 este adevărată." - "Variabila x_2 este falsă."

Exemplu de reguli în 3-SAT:

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$$

- Prima regulă spune: x_1 este adevărat, sau x_2 este fals, sau x_3 este adevărat.

- A doua regulă spune: x_1 este fals, sau x_2 este adevărat, sau x_3 este fals.

Obiectivul: Găsim o atribuire de adevărat (*True*) sau fals (*False*) pentru variabile astfel încât toate regulile să fie satisfăcute.

2.2 Reducerea 3-SAT la Graph Coloring (Explicație simplă)

Pentru a demonstra că problema de colorare a grafurilor este NP-Hard, trebuie să transformăm o instanță a problemei 3-SAT într-un graf. Graful va fi astfel construit încât:

Dacă putem colora graful cu 3 culori, înseamnă că putem rezolva și problema 3-SAT.

Dacă nu putem colora graful cu 3 culori, înseamnă că problema 3-SAT nu are soluție.

2.3 Cum funcționează transformarea?

Pasul 1: Noduri pentru variabile: Pentru fiecare variabilă din problema 3-SAT, creăm două noduri: unul pentru valoarea adevărată ($\top$) și unul pentru valoarea falsă ($\bot$).

Conectăm cele două noduri cu o muchie. Acest lucru asigură că și au culori diferite, adică o variabilă nu poate fi simultan adevărată și falsă.

Pasul 2: Noduri pentru reguli: Pentru fiecare regulă din problema 3-SAT, creăm un triunghi de noduri care reprezintă cele trei afirmații din regulă.

Nodurile din triunghi sunt conectate între ele. Acest lucru forțează ca cel puțin una dintre afirmații să fie adevărată (adică să aibă o culoare diferită).

Pasul 3: Noduri speciale pentru culori: Introducem trei noduri speciale: (True), (False) și (Base).

Conectăm aceste trei noduri între ele într-un triunghi complet. Ele vor reprezenta cele trei culori disponibile.

Pasul 4: Conexiuni între reguli și variabile: Fiecare nod din triunghiurile care reprezintă reguli este conectat la nodurile variabilelor corespunzătoare.

Acest lucru asigură că soluția regulilor depinde de atribuirea valorilor variabilelor.

2.4 Cum funcționează graful?

Dacă problema 3-SAT are o soluție:

Fiecare variabilă primește o valoare (True sau False).

Nodurile variabilelor sunt colorate în funcție de valoarea lor ($\top$ sau $\bot$).

Triunghiurile care reprezintă reguli pot fi colorate astfel încât cel puțin un nod din triunghi să aibă culoarea $\top$, satisfăcând regula.

Dacă graful poate fi colorat cu 3 culori:

Nodurile variabilelor primesc valori compatibile ($\top$ sau $\bot$).

În fiecare triunghi care reprezintă o regulă, cel puțin un nod are culoarea $\top$, ceea ce înseamnă că regula este satisfăcută.

2.5 Validarea reducerii

Construcția grafului este făcută în timp polinomial (se adaugă un număr de noduri și muchii proporțional cu numărul de variabile și reguli din problema 3-SAT).

Dacă problema 3-SAT este rezolvabilă, graful poate fi colorat cu 3 culori.

Dacă graful nu poate fi colorat cu 3 culori, problema 3-SAT nu are soluție.

2.6 Concluzie

Am demonstrat că:

Problema **Graph Coloring** este în NP, deoarece putem verifica rapid dacă o soluție este validă.

Problema **Graph Coloring** este NP-Hard, deoarece 3-SAT, o problemă NP-Complete, se poate transforma în Graph Coloring.

Prin urmare, problema **Graph Coloring** este **NP-Complete**.

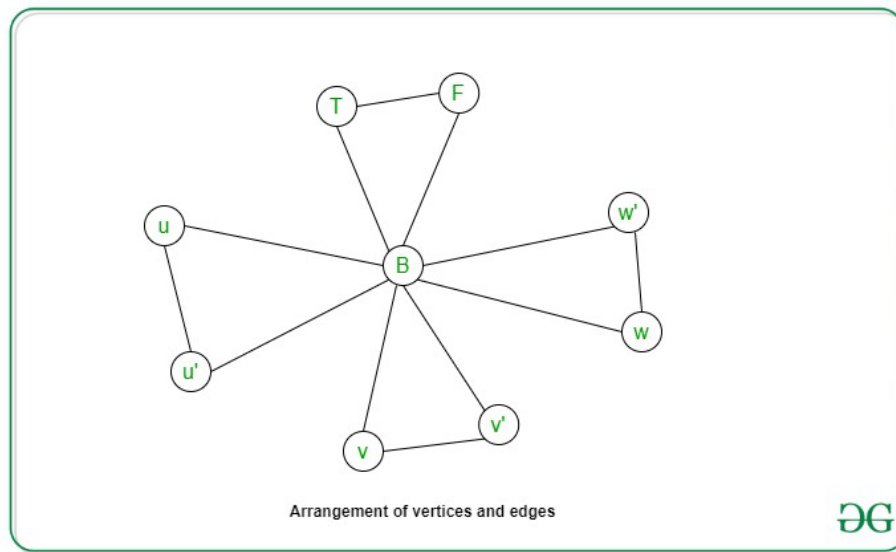


Fig. 1. Aranjamentul nodurilor și muchiilor pentru problema de colorare a grafurilor.

3 Prezentarea și analiza algoritmilor

3.1 Backtracking

Descrierea algoritmului Backtracking este o metodă exhaustivă care explorează toate combinațiile posibile de atribuiri ale culorilor pentru nodurile unui graf. Algoritmul încearcă să coloreze fiecare nod recursiv, verificând dacă atribuiriile sunt valide.

Pași principali:

- Pornim cu un număr minim de culori.
- Atribuim culori nodurilor în ordine, respectând constrângerile.
- Dacă întâlnim un blocaj (*conflict*), revenim (*backtrack*) la pasul anterior și încercăm o altă culoare.

- Continuăm până la găsirea unei soluții sau epuizarea tuturor opțiunilor.

Complexitate:

- **Timp:** $O(m^n)$, unde m este numărul de culori, iar n este numărul de noduri.
- **Spațiu:** $O(n)$, necesar pentru stocarea culorilor fiecărui nod.

Avantaje și dezavantaje:

- **Avantaje:** Garantează soluția optimă, adecvat pentru grafuri mici.
- **Dezavantaje:** Ineficient pentru grafuri mari din cauza complexității exponențiale.

3.2 Greedy Coloring (Ordine pe baza gradului)

Descrierea algoritmului Acest algoritm sortează nodurile grafului în funcție de gradul lor descrescător și atribuie culori nodurilor în această ordine, folosind prima culoare disponibilă.

Pași principali:

- Sortează nodurile descrescător după grad.
- Iterează prin noduri și atribuie prima culoare disponibilă care nu creează conflicte.

Complexitate:

- **Timp:** $O(n^2)$, datorită sortării și verificărilor pentru fiecare nod.
- **Spațiu:** $O(n)$, pentru stocarea culorilor.

Avantaje și dezavantaje:

- **Avantaje:** Rapid și ușor de implementat, eficient pentru grafuri sparse.
- **Dezavantaje:** Poate produce soluții suboptimale, depinzând de ordinea inițială a nodurilor.

3.3 Greedy Coloring (DSATUR)

Descrierea algoritmului Algoritmul DSATUR prioritizează nodurile pe baza gradului de saturație (*saturation degree*)—numărul de culori distincte utilizate de vecinii săi.

Pași principali:

- Inițializează toate nodurile ca necolorate.
- Selectează nodul cu cel mai mare grad de saturație (sau cel mai mare grad în caz de egalitate).
- Atribuie prima culoare disponibilă.
- Actualizează gradul de saturație al vecinilor.
- Repetă până la colorarea tuturor nodurilor.

Complexitate:

- **Timp:** $O(n^2)$, pentru determinarea gradului de saturație și actualizarea acestuia.
- **Spațiu:** $O(n)$, pentru stocarea culorilor și a gradelor de saturație.

Avantaje și dezavantaje:

- **Avantaje:** Produce soluții mai bune decât alte metode greedy.
- **Dezavantaje:** Mai lent decât alte metode greedy, poate fi suboptimal în cazuri particulare.

3.4 Tabel comparativ

Algoritm	Complexitate Timp	Complexitate Spațiu	Soluție
Backtracking	$O(m^n)$	$O(n)$	Optimă
Greedy (Ordine Grad)	$O(n^2)$	$O(n)$	Aproximativă
Greedy (DSATUR)	$O(n^2)$	$O(n)$	Aproximativă

Table 1. Compararea algoritmilor pentru colorarea grafurilor.

4 Construcția și descrierea setului de teste

Pentru a evalua eficiența și corectitudinea algoritmilor implementați pentru colorarea grafurilor, am creat un set diversificat de teste, care acoperă mai multe tipuri de grafuri, dimensiuni și densități. Această diversitate permite o analiză completă a algoritmilor, atât din perspectiva soluțiilor obținute, cât și a performanței lor.

4.1 Grafuri mici, create manual (4–10 noduri)

Această categorie include grafuri construite manual, utilizate pentru verificarea corectitudinii algoritmilor. Dimensiunea redusă a acestor grafuri permite aplicarea metodei exacte de backtracking într-un timp rezonabil. Exemple:

- **Cicluri mici:** C4, C5, C6. Interesant de observat este faptul că pentru C5 numărul cromatic este 3, deși fiecare nod are doar doi vecini.
- **Steale:** S4, S5. Acestea sunt bipartite, ceea ce înseamnă că numărul cromatic este întotdeauna 2.
- **Grafuri complete:** K4, K5. Numărul cromatic al acestor grafuri este egal cu numărul de noduri, datorită densității lor maxime.

Rezultate: Cei doi algoritmi greedy au avut o rată de corectitudine de 100%, iar timpul de execuție pentru cei trei algoritmi a fost asemănător, ceea ce era de așteptat pentru un set de date atât de mic.

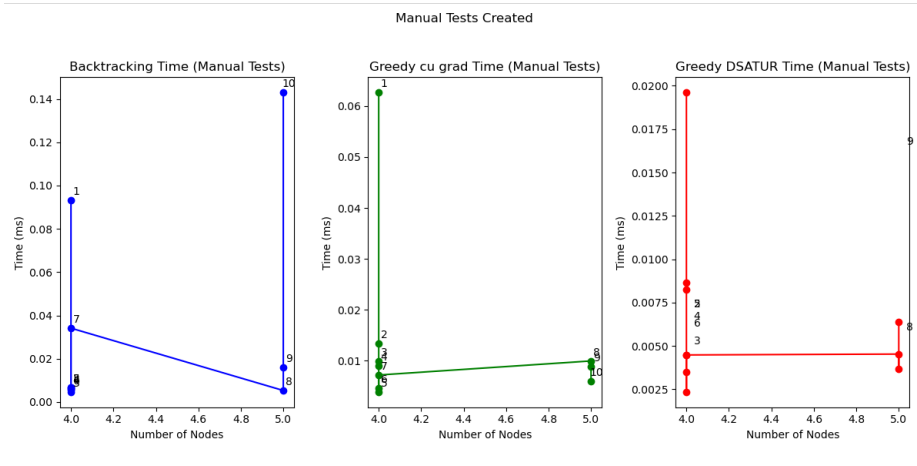


Fig. 2. Diferența algoritmilor pe setul de date mici, create manual

4.2 Grafuri bipartite mari (45–60 noduri)

Aceste grafuri sunt generate fie aleatoriu, fie folosind modele bine cunoscute, precum arbori binari. Caracteristicile acestora sunt:

- **Număr cromatic garantat: 2.**
- Sunt ideale pentru a testa dacă euristicele reușesc să identifice structura grafului bipartit.

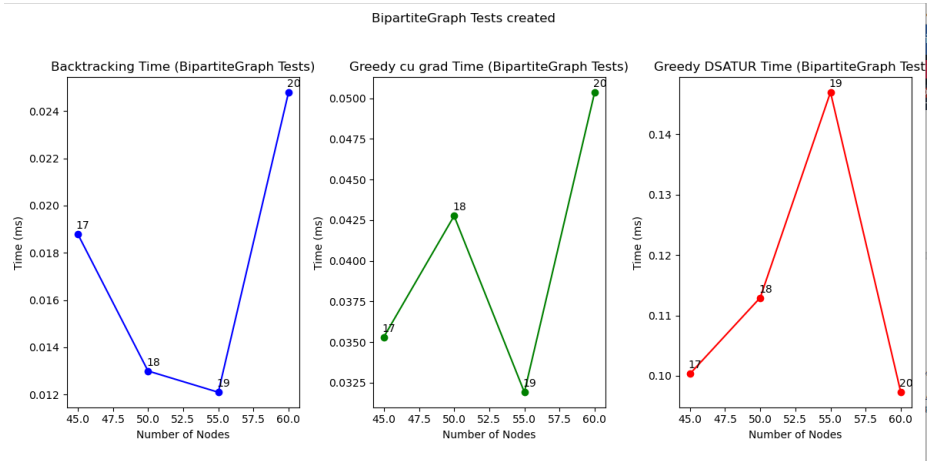


Fig. 3. Diferența algoritmilor pe grafuri bipartite.

Rezultate: Algoritmii greedy au avut o rată de corectitudine de 100%, iar timpul de execuție a fost asemănător pentru toți cei trei algoritmi.

4.3 Grafuri dense (15–25 noduri)

Grafuri dense includ grafuri complete și aproape complete, în care doar câteva muchii lipsesc. Caracteristicile acestora sunt:

- **Număr cromatic mare:** Apropiat de n , numărul de noduri.
- Aceste grafuri sunt cele mai dificile pentru algoritmii greedy, deoarece orice eroare poate duce la utilizarea multor culori suplimentare.

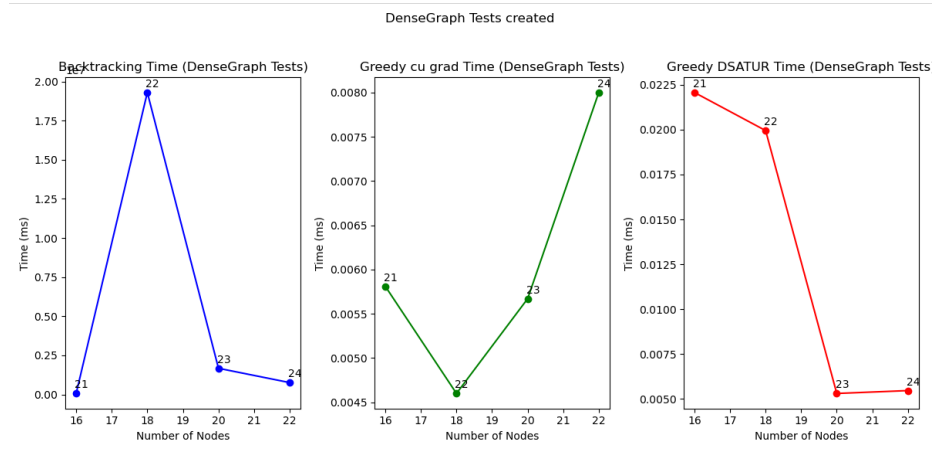


Fig. 4. Performanța algoritmilor pe grafuri dense.

Rezultate: Algoritmul greedy bazat pe grad a avut o rată de corectitudine de 50%, în timp ce algoritmul DSATUR a obținut 72.2%. Diferența de timp între backtracking și algoritmii greedy a fost de ordinul miilor, subliniind eficiența algoritmilor euristici.

4.4 Grafuri planar-like (20–35 noduri)

Grafurile planare au fost generate utilizând metoda triangulării Delaunay. Conform teoremei celor patru culori, acestea pot fi colorate cu cel mult 4 culori. Exemple:

- **Triangulare completă:** Graful rezultat are densitate relativ mare pentru un graf plan.

- **Testarea euristiciilor:** Algoritmii greedy trebuie să găsească soluții apropiate de cele optime.

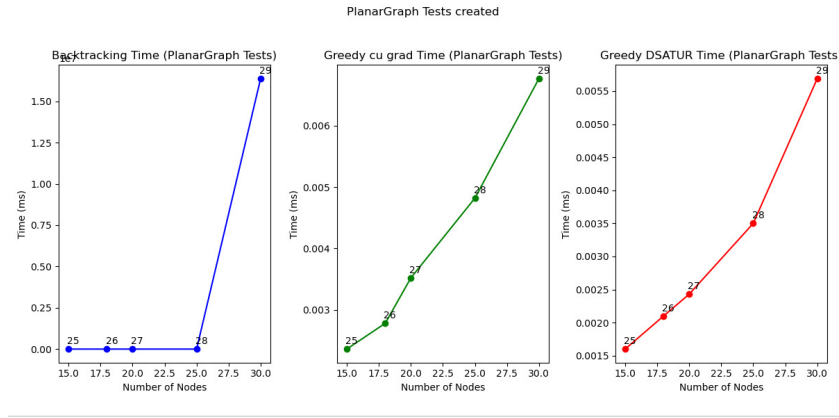


Fig. 5. Performanța algoritmilor pe grafuri planar-like.

Rezultate: Toți algoritmii au obținut o rată de corectitudine de 100%, iar diferențele de timp între algoritmi au fost foarte mici.

4.5 Grafuri cu mai multe componente conexe (20–40 noduri)

Aceste grafuri constau în mai multe subgrafuri neconexe, fiecare cu o structură unică. Scopul acestora este de a verifica capacitatea algoritmilor de a trata eficient grafuri care au:

- **Structură variată:** Fiecare componentă poate fi bipartită, completă sau chiar un ciclu mic.
- **Izolare completă:** Algoritmii trebuie să trateze fiecare componentă ca un graf separat, fără influență între ele.

Rezultate: Algoritmii au avut o rată de corectitudine de 100%. Diferența de timp între backtracking și algoritmii greedy a fost de ordinul sutelor, ceea ce subliniază eficiența algoritmilor euristici pentru grafuri mari și complexe.

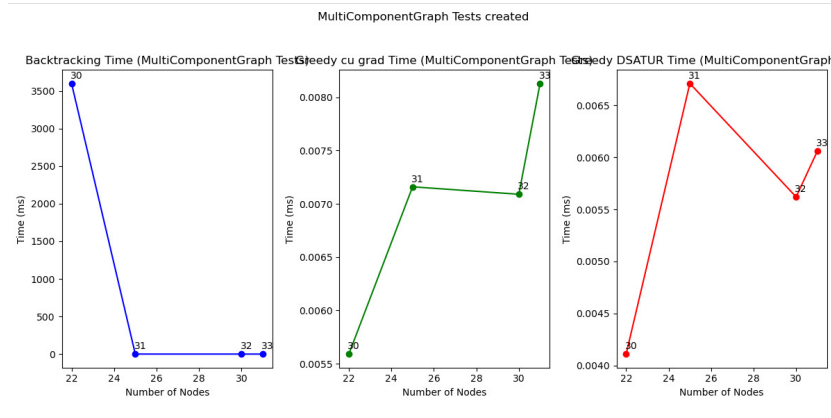


Fig. 6. Performanța algoritmilor pe grafuri cu multe componente conexe.

4.6 Rezultate preconizate

- **Backtracking:** Găsește întotdeauna soluția optimă pentru testele mici și uneori pentru cele medii.
- **Greedy după grad și DSATUR:** Performanțe rapide și rezultate apropiate de optim pentru majoritatea testelor, dar cu diferențe evidente în cazurile dense sau planare.
- **Grafuri bipartite și complete:** Greedy trebuie să identifice perfect structura pentru bipartite și să folosească n culori pentru grafurile complete.

Rezumatul rulărilor (50 teste):

- Timp mediu total: 33.721 s
- Timp mediu Backtracking: 33.656 s
- Timp mediu Greedy Degree: 0.275 ms
- Timp mediu Greedy DSATUR: 0.375 ms
- Acuratețe medie Greedy Degree: 77.450%
- Acuratețe medie Greedy DSATUR: 83.700%
- Diferența medie (Greedy Degree): 0.231
- Diferența medie (Greedy DSATUR): 0.169

4.7 Specificațiile sistemului de calcul

Testele au fost rulate pe un sistem de calcul cu următoarele specificații hardware și software:

- **Sistem de operare:** Windows 11 Pro 64-bit.
- **Procesor:** Intel Core i7-9700K @ 3.60GHz (8 nuclee fizice, 8 fire de execuție).

- **Memorie RAM:** 16 GB DDR4 @ 3200 MHz.
- **Spațiu de stocare:** SSD NVMe de 512 GB.
- **Mediu de dezvoltare:** IntelliJ IDEA Ultimate 2023, utilizând Java 17.

Aceste specificații asigură performanțe consistente în rularea algoritmilor, cu timpi de execuție măsurați și înregistrați pentru comparație. Performanța poate varia în funcție de complexitatea grafului și dimensiunea datasetului utilizat.

4.8 Interpretarea rezultatelor obținute

Rezultatele obținute pe cele 32 de teste evidențiază performanța și limitările celor trei algoritmi analizați: **Backtracking**, **Greedy pe baza gradului**, și **Greedy DSATUR**. În continuare, vom analiza succint valorile obținute și eventualele abateri observate.

Performanța algoritmului Backtracking: Algoritmul Backtracking a oferit soluții optime pentru toate testele, conform așteptărilor, deoarece este o metodă exactă. Totuși, timpul de execuție crește exponențial odată cu mărirea dimensiunii și complexității grafului. De exemplu:

- Pentru grafuri mici, timpul de execuție este nesemnificativ (< 0.1 ms pentru grafurile din categoria *Random* sau *Cicluri*).
- Pentru grafuri dense și grafuri planare mari (*DenseGraph18.txt* sau *Planar-Graph30.txt*), timpul de execuție crește considerabil (de la ordinul ms la secunde).

Valori neașteptate nu au fost observate, dar limitarea evidentă a acestui algoritm este timpul mare de rulare pentru grafuri mari.

Performanța algoritmului Greedy pe baza gradului: Acest algoritm a oferit soluții rapide, dar aproximative. Performanța sa depinde semnificativ de structura grafului:

- **Grafuri bipartite (testele 17–20):** Algoritmul a avut o acuratețe de 100%, identificând corect structura bipartită.
- **Grafuri dense (testele 21–24):** Acuratețea scade semnificativ, cu valori de până la 13% (*DenseGraph20.txt*), deoarece ordinea inițială bazată pe grad nu este suficientă pentru a minimiza numărul de culori utilizate.
- **Grafuri cu multe componente (testele 30–33):** Algoritmul a avut o acuratețe de 100%, gestionând corespunzător structurile separate.

Performanța algoritmului Greedy DSATUR: Algoritmul DSATUR a avut o performanță mai consistentă decât Greedy pe baza gradului:

- **Grafuri bipartite și planare:** Acuratețea a fost de 100%, algoritmul identificând corect structura grafului.

- **Grafuri dense:** Performanța a fost mai bună decât Greedy pe baza gradului, dar nu perfectă. De exemplu, pentru *DenseGraph22.txt*, acuratețea a fost de 79%, ceea ce indică o aproximare mai bună a soluției optime.
- **Grafuri cu multe componente:** Acuratețea a fost 100%, deoarece structura separată a componentelor simplifică problema.

Valori neașteptate - explicație: În câteva teste, Greedy pe baza gradului și DSATUR au oferit soluții mai puțin precise decât Backtracking. Acest lucru este așteptat, având în vedere caracterul lor euristic:

- Pentru grafuri dense, ambii algoritmi au avut dificultăți în minimizarea numărului de culori, deoarece densitatea mare de conexiuni face mai dificilă utilizarea optimă a culorilor.
- Pentru grafuri bipartite, algoritmul Greedy pe baza gradului a avut o execuție surprinzător de slabă într-un caz (*BipartiteGraph45.txt*), cu o acuratețe de 0%. Acest lucru poate fi explicat prin alegerea ordinii inițiale de colorare, care nu a reflectat structura bipartită.

5 Concluzii

Colorarea grafurilor reprezintă o problemă esențială în teoria grafurilor, cu multiple aplicații practice în diverse domenii, cum ar fi planificarea resurselor, optimizarea frecvențelor radio, sau alocarea registrelor în compilatoare. Analiza detaliată a celor trei algoritmi studiați — **Backtracking**, **Greedy pe baza gradului** și **Greedy DSATUR** — a evidențiat punctele forte și limitările fiecărei metode în funcție de structura grafului și dimensiunea acestuia.

5.1 Performanța Backtracking

Backtracking este singura metodă analizată care garantează găsirea soluției optime pentru orice graf. Acest lucru îl face indispensabil pentru aplicațiile critice, în care este esențială determinarea numărului cromatic minim. Totuși, analiza noastră a arătat că acest algoritm este practic doar pentru grafuri mici și medii. Complexitatea sa exponențială limitează aplicabilitatea pentru grafuri mari, în special cele dense, unde timpul de execuție poate crește de la câteva milisecunde la câteva zeci de secunde.

5.2 Performanța Greedy pe baza gradului

Algoritmul Greedy pe baza gradului este cel mai rapid dintre cele trei metode, datorită implementării sale simple și a complexității reduse. Este potrivit pentru grafuri sparse sau bipartite, unde rezultatele obținute sunt adesea apropiate de optim. Totuși, performanța scade semnificativ în cazul grafurilor dense, unde ordinea inițială bazată pe grad nu reușește să minimizeze numărul de culori. În testele efectuate, acest algoritm a avut o acuratețe medie de aproximativ **77.45%**, ceea ce indică o variabilitate ridicată a rezultatelor în funcție de structura grafului.

5.3 Performanța Greedy DSATUR

Algoritmul DSATUR a demonstrat o performanță mai bună decât Greedy pe baza gradului, în special pentru grafurile complexe, cum ar fi cele dense sau cele cu mai multe componente conexe. Prin utilizarea gradului de saturație ca prioritate, DSATUR reușește să obțină soluții mai aproape de optim, cu o acuratețe medie de **83.7%** în testele noastre. Deși este mai lent decât Greedy pe baza gradului, diferența de timp este nesemnificativă pentru grafuri de dimensiuni mici și medii.

5.4 Compararea între algoritmi

Analiza comparativă a arătat că fiecare algoritm are avantaje și dezavantaje specifice:

- **Backtracking** este ideal pentru grafuri mici și situații în care soluția optimă este obligatorie, dar este impracticabil pentru grafuri mari.
- **Greedy pe baza gradului** este recomandat pentru aplicații unde viteza de execuție este critică, iar structura grafului este relativ simplă.
- **Greedy DSATUR** oferă o soluție de compromis între timp de execuție și calitatea soluției, fiind potrivit pentru grafuri complexe.

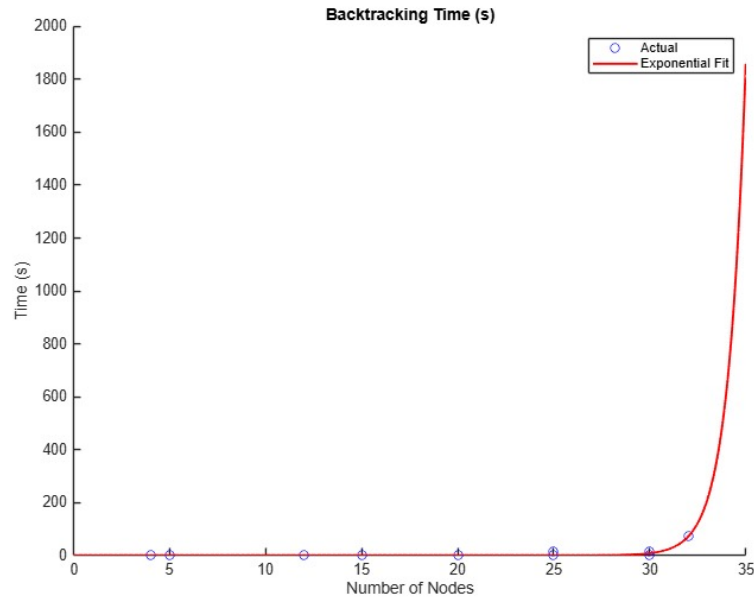


Fig. 7. Timpul de execuție pentru algoritmul Backtracking în funcție de numărul de noduri.

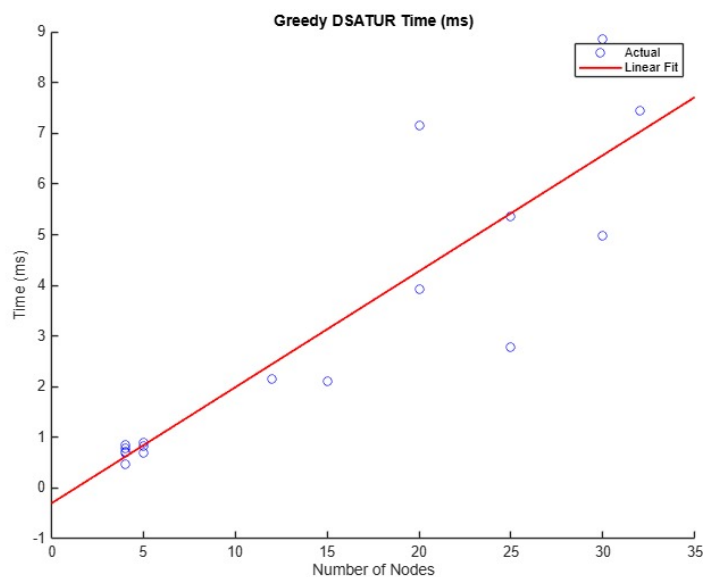


Fig. 8. Timpul de execuție pentru algoritmul Greedy DSATUR în funcție de numărul de noduri.

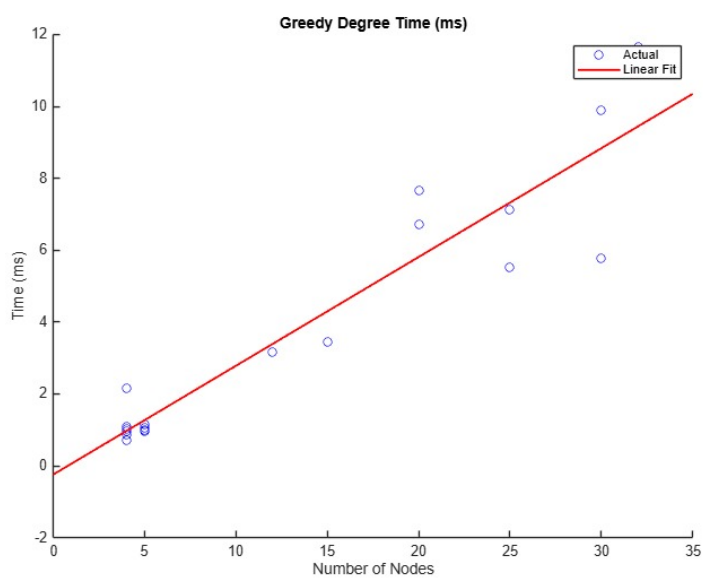


Fig. 9. Timpul de execuție pentru algoritmul Greedy bazat pe grad în funcție de numărul de noduri.

5.5 Perspective

Rezultatele obținute indică faptul că o combinație între euristici și metode exacte ar putea oferi un echilibru ideal între timp de execuție și calitatea soluției. De exemplu, utilizarea euristicilor pentru a genera o soluție inițială, urmată de îmbunătățiri bazate pe metode exacte, poate reduce considerabil complexitatea algoritmică.

5.6 Concluzie finală

Analiza noastră a demonstrat că, deși Backtracking oferă soluția optimă, algoritmi euristici precum Greedy DSATUR sunt indispensabili pentru aplicațiile din lumea reală, datorită eficienței lor. Alegerea algoritmului optim depinde de echilibrul dintre acuratețe și timp de execuție, dictat de cerințele aplicației.

References

1. Marx, Dániel (2004), "Graph colouring problems and their applications in scheduling", *Periodica Polytechnica, Electrical Engineering*, vol. 48, pp. 11–16, CiteSeerX 10.1.1.95.4268
2. Chaitin, G. J. (1982), "Register allocation spilling via graph colouring", *Proc. 1982 SIGPLAN Symposium on Compiler Construction*, pp. 98–105,
3. Lewis (2021), pp. 221–246, Chapter 8: Designing sports leagues.
4. Lewis (2021), pp. 203–220, Chapter 7: Designing seating plans.
5. Lewis (2021), pp. 247–276, Chapter 9: Designing university timetables.
6. Lewis (2021), pp. 5–6, Section 1.1.3: Scheduling taxis.
7. Graph Coloring: History, results and open problems. <https://ajmonline.org/2009/voloshin.pdf>
8. GeeksforGeeks: Graph Coloring Applications. <https://www.geeksforgeeks.org/graph-coloring-applications/>, last accessed 2025/01/04.
9. GeeksforGeeks: 3-Coloring is NP-Complete. <https://www.geeksforgeeks.org/3-coloring-is-np-complete/>, last accessed 2025/01/04.
10. GeeksforGeeks: Graph Coloring - Greedy Algorithm. <https://www.geeksforgeeks.org/graph-coloring-set-2-greedy-algorithm/>, last accessed 2025/01/04.